whereas a low discount rate leads to preferring only short-term time horizons.

A derived concept from this basic MDP framework is that of a *value function*. A value function assigns a value to either states or state-action pairs (i.e., a value for taking an action in a given state), with the former being called a *state-value function* (or often just the value function) and the latter being the *action value* or *Q function*. The value of a state is simply the expected return given that the agent starts in that state and follows some policy π, so value functions implicitly depend on the policy. Similarly, the action value or *Q value* of a state-action pair is the expected return given that the agent takes the action in that state and follows the policy π until the end. A state that puts the agent in close position to win a game, for example, would be assigned a high state value assuming the underlying policy was reasonable. We denote the value function as $V_\pi(s)$, with the subscript π indicating the dependence of the value on the underlying policy, and the Q function as $Q_\pi(s,a)$, although we often drop the π subscript for convenience.

Right now we understand the function $Q_\pi(s,a)$ as being some sort of black box that tells us the exact expected rewards for state action a in state s, but, of course, we do not have access to such an all-knowing function; we have to estimate it. In this book we used *neural networks* to estimate value functions and policy functions, although any suitable function could work. In the case of a neural-based $Q_\pi(s,a)$, we trained neural networks to predict the expected rewards. The value functions are defined and approximated recursively, such that $Q_\pi(s,a)$ is updated as

$$V_\pi(s) = r_t + \gamma V_\pi(s')$$

where s' refers to the next state, or s_{t+1}. For example, in Gridworld, landing on the goal tile results in +10, landing on the pit results in –10, and losing the game, and all other non-terminal moves, are penalized at –1. If the agent is two steps away from the winning goal tile, the final state reduces to $V_\pi(s_3) = 10$. Then, if $\gamma = 0.9$, the previous state is valued at $V_\pi(s_2) = r_2 + 0.9V_\pi(s_3) = –1 + 9 = 8$. The move before that must be $V_\pi(s_1) = r_1 +$